

数值计算方法 第二次上机作业

自动化（控制）

1 问题描述

编写不动点迭代算法程序，求解非线性方程 $x^3 - x^2 - 1 = 0$ 在区间 $[1.3, 1.6]$ 的一个实根。分别采用以下四种迭代方式，记录迭代过程并分析不同收敛性的原因：

1. $x_{k+1} = \phi_1(x_k) = \frac{1}{\sqrt{x_k - 1}}$
2. $x_{k+1} = \phi_2(x_k) = 1 + \frac{1}{x_k^2}$
3. $x_{k+1} = \phi_3(x_k) = (1 + x_k^2)^{1/3}$
4. $x_{k+1} = \phi_4(x_k) = 1 + \frac{x_k^2}{x_k^2 + x_k + 1}$

2 问题分析

本题求解方程 $f(x) = x^3 - x^2 - 1 = 0$ 。通过计算 $f(1.3) \approx -0.493$, $f(1.6) \approx 0.536$, 可知根位于 $[1.3, 1.6]$ 之间。不动点迭代法收敛的充分条件是在根 x^* 的邻域内满足 $|\phi'(x^*)| < 1$ 。

可以先估算根的近似值 $x^* \approx 1.46$, 然后计算各 $\phi'_i(x^*)$ 的绝对值来预测收敛性。通过初步计算各函数的导数, 我们可以预测不同迭代格式的收敛性:

- 方式 (1) $\phi_1(x) = (x - 1)^{-1/2}$ 导数绝对值大于 1, 预计发散。
- 方式 (2) $\phi_2(x) = 1 + x^{-2}$ 导数绝对值约为 0.64, 预计收敛但较慢。
- 方式 (3) $\phi_3(x) = (1 + x^2)^{1/3}$ 导数绝对值约为 0.46, 预计收敛且比方式 (2) 快。
- 方式 (4) $\phi_4(x) = 1 + \frac{x^2}{x^2 + x + 1}$ 导数绝对值约为 0.24, 预计收敛且最快。

不同的 $\phi(x)$ 构造方式决定了其导数绝对值的大小, 进而影响迭代的收敛性与收敛速度。本次实验将通过 MATLAB 编程验证不同格式的数值表现。

3 方式 (1)

3.1 算法分析

迭代格式为 $\phi_1(x) = (x-1)^{-1/2}$ 。其导数为 $\phi_1'(x) = -\frac{1}{2}(x-1)^{-3/2}$ 。由于 $|\phi_1'(x^*)| > 1$ ，理论预测该迭代格式发散。

3.2 MATLAB 程序

```
% 核心迭代逻辑 (完整代码附在报告最后)
phi1 = @(x) 1 / sqrt(x - 1);
x_next = phi1(x_curr);
err = abs(x_next - x_curr);
```

3.3 运行结果及分析

```
>>> 正在执行 方式(1): phi = 1/sqrt(x-1)
Iter      x_n          Error
0         1.50000000    ---
1         1.41421356    8.58e-02
...
7         0.88020425    1.41e+00
[警告] 迭代发散! 数值已失效或跳出搜索范围。
[结论] 迭代失败, 未能收敛。
```

分析: 实验结果显示, 方式 (1) 在第 7 步迭代后 x_n 减小至 0.88, 跳出了函数定义域并导致后续产生负数开方。这证实了导数绝对值大于 1 会导致误差放大并最终发散。

4 方式 (2)

4.1 算法分析

迭代格式为 $\phi_2(x) = 1 + x^{-2}$, 导数为 $\phi_2'(x) = -2x^{-3}$ 。在根处 $|\phi_2'(x^*)| \approx 0.64 < 1$ 。理论预测收敛, 且由于导数为负, 迭代序列将呈现摆动收敛特征。

4.2 MATLAB 程序

```
phi2 = @(x) 1 + 1 / (x^2);
x_next = phi2(x_curr);
```

4.3 运行结果及分析

```
>>> 正在执行 方式(2): phi = 1 + 1/x^2
Iter      x_n          Error
0         1.50000000    ---
1         1.44444444    5.56e-02
...
36         1.46557123    7.02e-09
[结论] 迭代成功收敛。最终根: 1.46557123, 共用 36 步。
```

分析: 方式 (2) 成功收敛, 但由于导数绝对值较大 (~ 0.64), 收敛速度较慢, 共需 36 步才达到容差要求。

5 方式 (3)

5.1 算法分析

迭代格式为 $\phi_3(x) = (1+x^2)^{1/3}$, 导数为 $\phi'_3(x) = \frac{2x}{3(1+x^2)^{2/3}}$ 。在根处 $|\phi'_3(x^*)| \approx 0.46 < 1$ 。理论预测收敛速度优于方式 (2)。

5.2 MATLAB 程序

```
phi3 = @(x) (1 + x^2)^(1/3);
x_next = phi3(x_curr);
```

5.3 运行结果及分析

```
>>> 正在执行 方式(3): phi = (1+x^2)^(1/3)
Iter      x_n          Error
0         1.50000000    ---
1         1.48124803    1.88e-02
...
20         1.46557124    5.95e-09
[结论] 迭代成功收敛。最终根: 1.46557124, 共用 20 步。
```

分析: 实验结果显示方式 (3) 仅需 20 步即可收敛。由于导数值为正, 迭代过程呈现出单调趋近根的特点。

6 方式 (4)

6.1 算法分析

迭代格式 $\phi_4(x) = 1 + \frac{x^2}{x^2+x+1}$, 导数为 $\phi'_4(x) = \frac{x^2+2x}{(x^2+x+1)^2}$ 。在根处 $|\phi'_4(x^*)| \approx 0.24$ 。这是四个格式中导数绝对值最小的一个, 预计收敛最快。

6.2 MATLAB 程序

```
phi4 = @(x) 1 + (x^2) / (x^2 + x + 1);  
x_next = phi4(x_curr);
```

6.3 运行结果及分析

```
>>> 正在执行 方式(4): phi = 1 + x^2/(x^2+x+1)  
Iter    x_n          Error  
0       1.50000000    ---  
1       1.47368421    2.63e-02  
...  
12      1.46557123    3.68e-09  
[结论] 迭代成功收敛。最终根: 1.46557123, 共用 12 步。
```

分析: 方式 (4) 仅用 12 步即完成收敛, 效率最高。实验结果验证了“导数绝对值越小, 收敛速度越快”的理论。

7 比较总结

将四种迭代方式的实验数据汇总如下表:

方法	$ \phi'(x^*) $ (理论值)	迭代次数	最终结果	收敛性
方式 (1)	1.57	/	NaN	发散
方式 (2)	0.64	36	1.46557123	收敛
方式 (3)	0.46	20	1.46557124	收敛
方式 (4)	0.24	12	1.46557123	收敛

表 1: 不同迭代格式性能对比汇总

对比分析:

- 收敛条件:** 不动点迭代的收敛性严格取决于 $|\phi'(x)| < 1$ 。方式 (1) 因违反此条件而导致数值溢出, 这也提醒我们在构造迭代格式时必须首先进行导数估计。
- 收敛速度:** 收敛速度受 $|\phi'(x^*)|$ 大小支配。方式 (4) 通过更复杂的代数变形, 将导

数值降至 0.24 左右，从而在步数上比方式 (2) 节省了 66% 的计算量。

3. **数值特征：**观察运行数据可见，方式 (2) 的误差项在迭代中虽然减小但带有正负号波动（摆动收敛），而方式 (3)(4) 则是单调减小。

8 附：完整 MATLAB 程序及运行结果

8.1 完整 MATLAB 程序

```
clear; clc;

x0 = 1.5;
tol = 1e-8;
max_iter = 50;

phi1 = @(x) 1 / sqrt(x - 1);
phi2 = @(x) 1 + 1 / (x^2);
phi3 = @(x) (1 + x^2)^(1/3);
phi4 = @(x) 1 + (x^2) / (x^2 + x + 1);

phis = {phi1, phi2, phi3, phi4};
names = {'方式(1): phi = 1/sqrt(x-1)', ...
         '方式(2): phi = 1 + 1/x^2', ...
         '方式(3): phi = (1+x^2)^(1/3)', ...
         '方式(4): phi = 1 + x^2/(x^2+x+1)'};

for i = 1:4
    fprintf('\n>>> 正在执行 %s\n', names{i});
    fprintf('%-8s %-15s %-15s\n', 'Iter', 'x_n', 'Error');
    x_curr = x0;
    iter = 0;
    converged = false;
    fprintf('%-8d %-15.8f %-15s\n', iter, x_curr, '---');

    for k = 1:max_iter
        try
            x_next = phis{i}(x_curr);
        catch
            fprintf(' [错误] 计算过程中出现数学异常\n');
            break;
        end
        if isnan(x_next) || isinf(x_next) || ~isreal(x_next) || abs(x_next) > 10
            fprintf(' [警告] 迭代发散! 数值已失效.\n');
            break;
        end
    end
end
```

```
end
err = abs(x_next - x_curr);
x_curr = x_next;
iter = k;
fprintf('%-8d %-15.8f %-15.2e\n', iter, x_curr, err);
if err < tol
    converged = true;
    break;
end
end
if converged
    fprintf(' [结论] 迭代成功收敛。最终根：%.8f，共用 %d 步。\\n', x_curr, iter
);
else
    fprintf(' [结论] 迭代失败，未能收敛。\\n');
end
end
```

8.2 完整运行结果

(此处省略部分重复的中间迭代步数，以保持报告简洁)

```
>>> 正在执行 方式(1) ... [警告] 迭代发散！数值已失效。
>>> 正在执行 方式(2) ... [结论] 迭代成功收敛。最终根：1.46557123，共用 36 步。
>>> 正在执行 方式(3) ... [结论] 迭代成功收敛。最终根：1.46557124，共用 20 步。
>>> 正在执行 方式(4) ... [结论] 迭代成功收敛。最终根：1.46557123，共用 12 步。
```